

EECS 31L
Lab 4: Datapath

Ali Mortada
19849621

March 9th, 2025

1 Objective

The purpose of this lab was to design an upper datapath module to connect the lower modules created in Lab 3, as well as design the data memory module which was not designed in Lab 3. Fig. 1 shows the block diagram for the RISC-V datapath. The instruction set supported by the processor remains the same as the one in Lab 3, shown in Fig. 2. However, two new instructions were added, **lw** and **sw**, which are shown in Figs. 3 and 4.

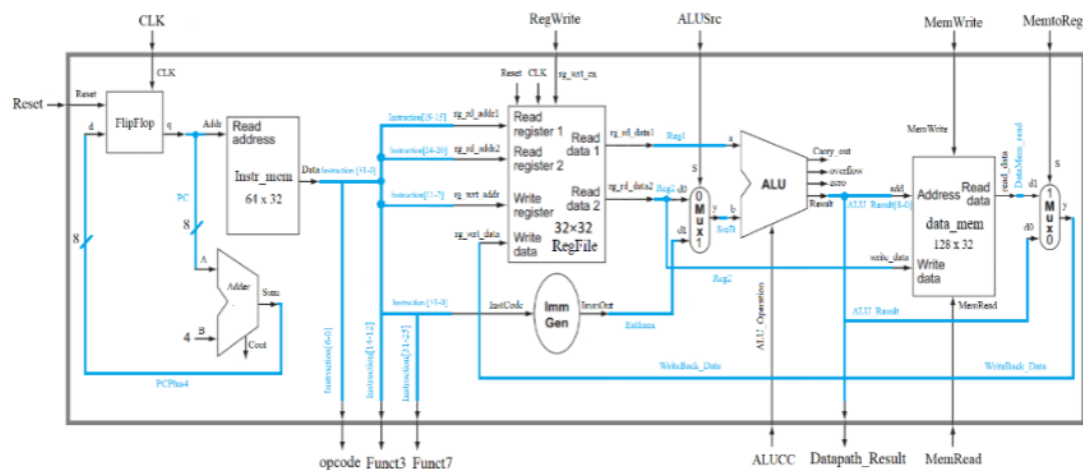


Figure 1: RISC-V datapath.

imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	100	rd	0010011	NORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	100	rd	0110011	NOR
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

Figure 2: RISC-V instruction set.

imm[11:0]	rs1	010	rd	0000011	LW
-----------	-----	-----	----	---------	----

Figure 3: Instruction set (lw).

imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
-----------	-----	-----	-----	----------	---------	----

Figure 4: Instruction set (sw).

2 Procedure

2.1 Instruction Memory

To account for the two new instructions provided in Figs. 3 and 4, two lines were added to the instruction memory module created in Lab 3. They are as follows:

```
memory[18] = 32'h02b02823; // sw r11, 48(r0) alu result = 32'h00000030
memory[19] = 32'h03002603; // lw r12, 48(r0) alu result = 32'h00000030
//      r12 = 32'h00000005
```

2.2 Data Memory

The data memory module is designed similarly to the instruction memory module of Lab 3. The main difference is that the processor has the ability to write to this memory as well as the ability to read from it. Thus, it has four inputs: **MemWrite**, the enable signal to write to memory, **MemRead**, the enable signal to read from memory, **addr**, which gives the address to read/write from, and **write_data**, the data to write to memory. It also has one output, **read_data**, the data to read from memory. Another main difference of the data memory file is that it is 128 by 32 bits, meaning that it can store twice the amount of data than the instruction memory. This memory is also byte-addressable. Fig. 5 shows the block diagram for the data memory.

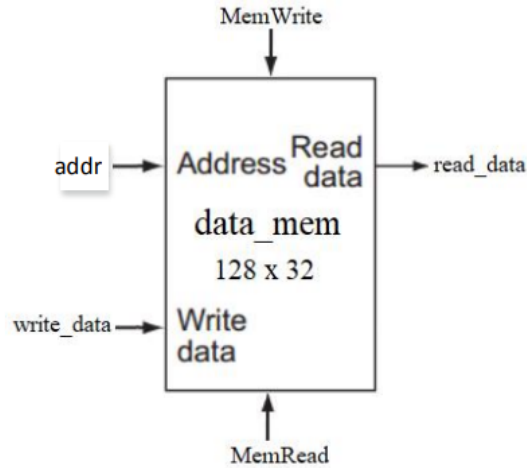


Figure 5: Data memory block diagram.

Since the memory is byte-addressable, only the 7 most significant bits from the `addr` input are used (`addr[8:2]`). The module's behavior was defined using the following code:

```
reg [31:0] memory [127:0]; // 128x32 internal memory

always @(*)
begin
    if (MemRead) read_data = memory[addr >> 2];
    if (MemWrite) memory[addr >> 2] = write_data;
end
```

First, the internal memory array is declared, similar to the instruction memory. Then, the `always` block defines the behavior of reading and writing to the memory; it checks if each control signal is high, and if so, performs the associated function. Since the memory is byte addressable, the address is shifted to the right by 2 bits when accessing the internal array.

2.3 Higher Level Module

At this point, all of the sub-modules had been designed and it was time to connect them to form the datapath module. Since the behavior of the processor was already defined in these sub-modules, all that was left was to declare the internal wires and connect the sub-modules to each other and to the inputs and outputs of the higher level module, using the block diagram in Fig. 1 as a guide. The following code was used to define the behavior of the module:

```
// define internal wires
wire [PC_W-1:0] PC;
wire [PC_W-1:0] PCPlus4 = PC + 4;
wire [INS_W-1:0] Instruction;
wire [DATA_W-1:0] WriteBack_Data;
wire [31:0] Reg1;
wire [31:0] Reg2;
wire [31:0] SrcB;
wire [31:0] ExtImm;
```

```

wire [31:0] ALU_Result;
wire [31:0] DataMem_read;
wire [RF_ADDRESS-1:0] rd_rg_wrt_wire;
wire [RF_ADDRESS-1:0] rd_rg_addr_wire1;
wire [RF_ADDRESS-1:0] rd_rg_addr_wire2;

assign rd_rg_wrt_wire = Instruction[11:7];
assign rd_rg_addr_wire1 = Instruction[19:15];
assign rd_rg_addr_wire2 = Instruction[24:20];

// instantiate inner modules and connect them to wires
FlipFlop ff(.clk(clk), .reset(reset), .d(PCPlus4), .q(PC));

InstMem inst_mem(.addr(PC), .instruction(Instruction));

RegFile reg_file(
    .clk(clk), .reset(reset), .rg_wrt_en(reg_write),
    .rg_wrt_addr(rd_rg_wrt_wire),
    .rg_rd_addr1(rd_rg_addr_wire1),
    .rg_rd_addr2(rd_rg_addr_wire2),
    .rg_wrt_data(WriteBack_Data),
    .rg_rd_data1(Reg1),
    .rg_rd_data2(Reg2)
);

ImmGen imm_gen(.InstCode(Instruction[31:0]), .ImmOut(ExtImm));

MUX21 mux1(.D1(Reg2), .D2(ExtImm), .S(alu_src), .Y(SrcB));

alu_32 alu(
    .A_in(Reg1),
    .B_in(SrcB),
    .ALU_Sel(alu_cc),
    .ALU_Out(ALU_Result),
    .Carry_Out(), .Zero(), .Overflow()
);

DataMem data_mem(
    .MemRead(mem_read),
    .MemWrite(mem_write),
    .addr(ALU_Result[DM_ADDRESS-1:0]),
    .write_data(Reg2),
    .read_data(DataMem_read)
);

MUX21 mux2(.D1(ALU_Result), .D2(DataMem_read), .S(mem2reg), .Y(WriteBack_Data));

// assign internal wires to module outputs
assign opcode = Instruction[6:0];
assign funct3 = Instruction[14:12];
assign funct7 = Instruction[31:25];

```

```
assign alu_result = ALU_Result;
```

3 Simulation Results

3.1 Datapath

The testbench code provided in the lab manual was used to test the datapath module. Below, in Fig. 6, is a screenshot of the simulation waveform:

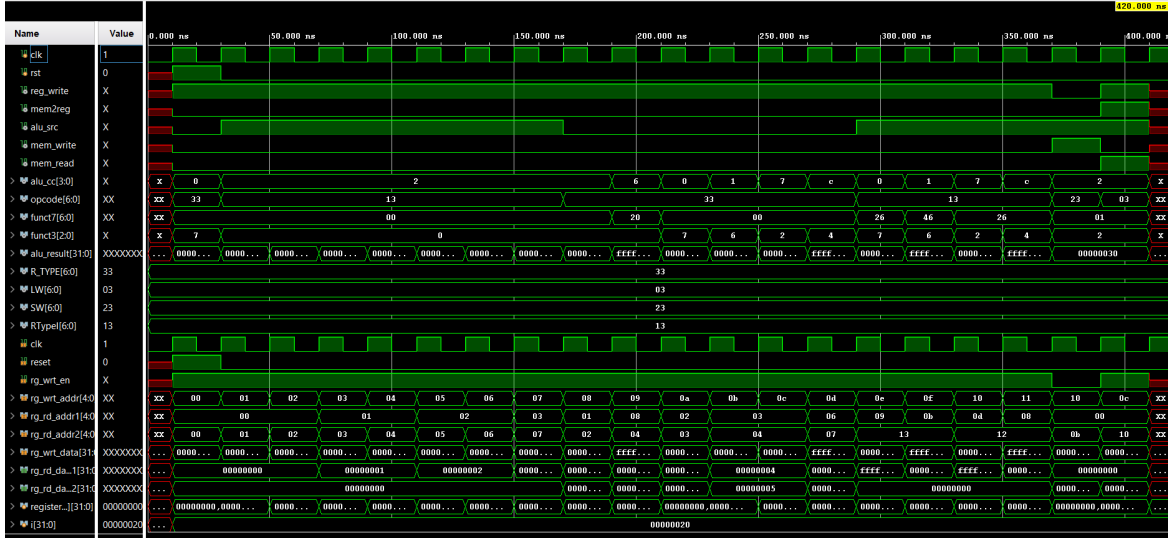


Figure 6: Datapath simulation waveform.

The waveform includes the inputs and outputs of the datapath module as well as the inputs and outputs of the register file module, which were added from the scope tab. It is clear that each of the signals matches those of the simulation waveform provided in the manual.